

Universidad Nacional de San Juan – Teoría de la Información

Práctico de Máquina 2:

Desafío de Proveedores de Compresión

Huffman dinámico vs. Brotli vs. gzip -6

Grupo 4: Carrascosa Daniel, Piñones Gustavo, Reus Lorenzo, Rueda Mariano, Terzano Jose, Zuliani Augusto

Catálogo de Soluciones: Propuesta de Valor

Desafío de Proveedores — Alternativas tecnológicas según el caso de uso frente al baseline

Huffman Dinámico (FGK)

Solución de Autoría Propia

- **Una sola pasada (streaming):** comprime sobre la marcha en tiempo real sin requerir el archivo completo de antemano.
- **Sin tablas en el archivo:** el árbol se adapta en espejo; cabecera fija de solo 20 B con verificación CRC32.
- **100% Python estándar:** desarrollado íntegramente por el grupo sin dependencias externas de compresión.

Brotli (Google)

Solución de Mercado (q5)

- **Estado del arte:** 15% a 25% más de compresión que gzip para datos generales y de la web.
- **Diccionario predefinido:** 120 KB estáticos de fábrica; reconoce palabras completas desde el primer instante.
- **Calidad 5 (sweet spot):** ventana de 4 MiB con balance óptimo entre ratio y velocidad de procesamiento.

gzip (Baseline)

Referencia de la Cátedra

- **Estándar universal:** punto de comparación clásico de la industria desde 1992 (DEFLATE: LZ77 + Huffman).
- **Nivel 6 por defecto:** el valor de referencia histórico entre compresión y tiempo de CPU.
- **Parámetro -n (no-name):** sin nombres ni marcas de tiempo; garantiza una evaluación puramente algorítmica.

Soluciones que ofrecemos

Solución	Tipo	Configuración	Método
Huffman dinámico FGK	Propia	Python 3.13, solo stdlib, formato .tdi	Huffman adaptativo, orden 0
Brotli	Tercerizada	brothli 1.1.0, -q 5	LZ77 + Huffman + contexto
gzip	Baseline de la cátedra	gzip 1.13, -n -6	LZ77 + Huffman (Deflate)

- Las tres se evalúan sobre los mismos archivos, en la misma máquina y con el mismo protocolo.

Corpus de pruebas

Archivo	Tipo	Tamaño	Símbolos	H_0 (bits)	$8 / H_0$
prueba_1	muy pequeño	64 B	14	3,31	2,42
prueba_2	texto natural	100 KiB	51	4,23	1,89
prueba_3	alta repetición	100 KiB	11	2,76	2,89
prueba_4	baja repetición	100 KiB	95	6,57	1,22

- H_0 : entropía empírica de bytes (orden 0).
- $8 / H_0$: ratio máximo que puede lograr cualquier código de orden 0, como Huffman.
- SHA-256 de los archivos verificado contra el publicado por la cátedra.

Huffman dinámico (algoritmo FGK)

- Compresor y descompresor arrancan con el mismo árbol: un solo nodo NYT (Not Yet Transmitted).
- Símbolo nuevo: se emite el código del NYT y el byte en crudo (8 bits).
- Símbolo conocido: se emite su camino raíz → hoja (0 izq., 1 der.).
- Después de cada símbolo los dos lados actualizan el árbol igual.
- No se guarda el árbol ni las frecuencias en el archivo.

Traza de prueba_1 (primeros 6 símbolos)

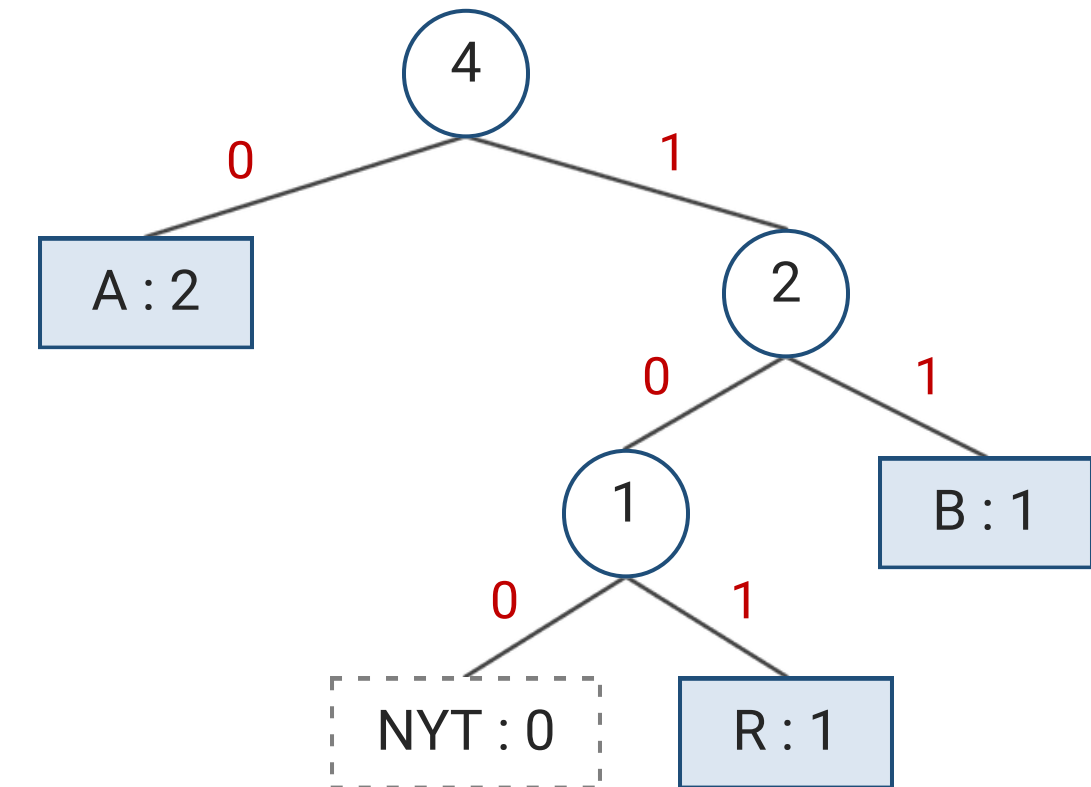
#	Byte	Tipo	Bits
0	A	nuevo	01000001
1	B	nuevo	0 01000010
2	R	nuevo	00 01010010
3	A	conocido	0
4	C	nuevo	100 01000011
5	A	conocido	0

Actualización del árbol

Desde la hoja hasta la raíz, en cada nodo:

1. Buscar el líder del bloque: el nodo de mayor número con el mismo peso (si es el padre, no se intercambia).
2. Intercambiar los subárboles.
3. Sumar 1 al peso y subir al padre.

Así se mantiene la **propiedad de hermanos**: pesos no decrecientes según el número de nodo, y el árbol sigue siendo de Huffman.



Árbol después de A, B, R, A (peso en cada nodo). Código del NYT: 100.

Formato .tdi: metadatos para descomprimir

Offset	Bytes	Campo	Para qué sirve
0	4	magic "TDI1"	Reconocer el formato
4	1	versión	Rechazar versiones incompatibles
5	1	algoritmo (1 = FGK)	Elegir el decodificador
6	1	flags	Reservado (0)
7	8	tamaño original	Saber cuándo parar (no hay símbolo EOF)
15	4	CRC32 del original	Detectar corrupción
19	1	bits válidos del último byte	Ignorar el relleno final

- Cabecera fija de 20 bytes. No se guarda árbol ni tabla de frecuencias.
- Overhead: 32 % en prueba_1 (64 B) y entre 0,02 % y 0,06 % en los archivos de 100 KiB.

Demostración en vivo: prueba 3 (alta repetición, 102400 B)

1. Solución propia (FGK), con traza

```
python compressor.py $orig demo\p3.tdi --trace  
python decompressor.py demo\p3.tdi demo\p3_propio.txt --verify $orig
```

2. gzip -n -6 (baseline)

```
Copy-Item $orig demo\p3_gzip.txt  
gzip -n -6 demo\p3_gzip.txt  
gzip -d -k demo\p3_gzip.txt.gz
```

3. Brotli q5 (tercerizada)

```
brotli.compress(datos, quality=5) → demo\p3.br  
brotli.decompress(comprimido) → demo\p3_brotli.txt
```

4. Integridad de las tres

```
Get-FileHash original, p3_propio, p3_gzip, p3_brotli -Algorithm SHA256
```

Resultado de la demostración

```
PS C:\Users\augus\OneDrive\Escritorio\Grupo4-TDI-TP-Compresor-main\Grupo4-TDI-TP-Compresor-main> dir demo

Directorio: C:\Users\augus\OneDrive\Escritorio\Grupo4-TDI-TP-Compresor-main\Grupo4-TDI-TP-Compresor-main\demo

Mode                LastWriteTime         Length Name
----                -
-a----            06/10/2026   13:18             63 p3.br
-a---l            06/10/2026   13:12          36661 p3.tdi
-a----            06/10/2026   13:18         102400 p3_brotli.txt
-a----            05/10/2026   19:04         102400 p3_gzip.txt
-a---l            05/10/2026   19:04           914 p3_gzip.txt.gz
-a---l            06/10/2026   13:16         102400 p3_propio.txt
```

```
PS C:\Users\augus\OneDrive\Escritorio\Grupo4-TDI-TP-Compresor-main\Grupo4-TDI-TP-Compresor-main> Get-FileHash $orig, demo\p3_propio.txt, demo\p3_gzip.txt, demo\p3_brotli.txt -Algorithm SHA256 | Format-List Hash, Path

Hash : A54F4A85A8695E1BEC7CF49603301D97BD08C89F98DDDB09FB0605FA1F88E36E
Path : C:\Users\augus\OneDrive\Escritorio\Grupo4-TDI-TP-Compresor-main\Grupo4-TDI-TP-Compresor-main\tests\corpus\prueba_3_alta_repeticion.txt

Hash : A54F4A85A8695E1BEC7CF49603301D97BD08C89F98DDDB09FB0605FA1F88E36E
Path : C:\Users\augus\OneDrive\Escritorio\Grupo4-TDI-TP-Compresor-main\Grupo4-TDI-TP-Compresor-main\demo\p3_propio.txt

Hash : A54F4A85A8695E1BEC7CF49603301D97BD08C89F98DDDB09FB0605FA1F88E36E
Path : C:\Users\augus\OneDrive\Escritorio\Grupo4-TDI-TP-Compresor-main\Grupo4-TDI-TP-Compresor-main\demo\p3_gzip.txt

Hash : A54F4A85A8695E1BEC7CF49603301D97BD08C89F98DDDB09FB0605FA1F88E36E
Path : C:\Users\augus\OneDrive\Escritorio\Grupo4-TDI-TP-Compresor-main\Grupo4-TDI-TP-Compresor-main\demo\p3_brotli.txt
```

Verificación de integridad

12 / 12

Reconstrucción idéntica

SHA256(original) =
SHA256(reconstruido) en las 3
soluciones × 4 archivos, en
todas las corridas del
benchmark.

2

Controles en la solución propia

- CRC32 guardado en la
cabecera (detecta un .tdi
dañado)
- SHA-256 contra el original
con la opción --verify.

29

Tests automáticos

Archivo vacío, 1 byte, un símbolo
repetido 10 000 veces, los 256
bytes, 100 KB aleatorios y bytes
alterados.

6

Errores detectados

Archivo inexistente, magic
inválido, versión no soportada,
cabecera truncada, datos
insuficientes, CRC distinto.

Metodología del benchmark

1. Se verifica el SHA-256 de los 4 archivos del corpus.
2. Cada herramienta se ejecuta como un proceso aparte: las tres pagan el arranque de su programa.
3. Por cada archivo y solución: 1 corrida de calentamiento (descartada) y 5 corridas medidas. Se informa la mediana.
4. En cada corrida se comprueba el SHA-256 del archivo reconstruido.
5. Los resultados se exportan a CSV, JSON y una tabla en Markdown, junto con los datos del entorno.

Equipo: AMD Ryzen 7 7735HS, Windows 11, Python 3.13, brotli 1.1.0, gzip 1.13.

Tabla comparativa

Archivo	Ratio FGK	Tiempo FGK	Ratio Brotli	Tiempo Brotli	Ratio gzip	Tiempo gzip
prueba_1 *	1,03	169 ms	1,33	16 ms	1,09	29 ms
prueba_2	1,88	410 ms	89,59	17 ms	53,36	28 ms
prueba_3	2,79	355 ms	1600,00	18 ms	112,04	22 ms
prueba_4	1,20	558 ms	1,20	27 ms	1,20	31 ms

- Ratio = División entre el tamaño original (So) y el comprimido (Sc).
- Tiempo de compresión.

Tabla comparativa Completa

Archivo	Solución	Ratio	Ahorro%	Vc (MB/s)	Vd (MB/s)
Prueba 1	Huffman dinámico FGK	1,032	3,12	0,000	0,000
Prueba 1	Brotli q5	1,333	25,00	0,004	0,004
Prueba 1	gzip -n -6	1,085	7,81	0,002	0,002
Prueba 2	Huffman dinámico FGK	1,878	46,75	0,250	0,207
Prueba 2	Brotli q5	89,589	98,88	5,931	3,617
Prueba 2	gzip -n -6	53,361	98,13	3,621	3,456
Prueba 3	Huffman dinámico FGK	2,793	64,20	0,289	0,251
Prueba 3	Brotli q5	1.600,000	99,94	5,716	2,698
Prueba 3	gzip -n -6	112,035	99,11	4,618	4,505
Prueba 4	Huffman dinámico FGK	1,199	16,95	0,183	0,159
Prueba 4	Brotli q5	1,204	16,95	3,862	3,861
Prueba 4	gzip -n -6	1,202	16,79	3,298	2,785

Weissman Score

$$W = (R / R_{ref}) \cdot (\ln T_{ref} / \ln T)$$

$$R_{global} = (\sum \text{tamaño original}) / (\sum \text{tamaño comprimido})$$

$$T_{global} = \text{mediana del tiempo total de compresión del corpus}$$

Solución	ΣSc	ΣSo	R global	T global	W
Brotli q5	86251 B	307200	3,562	60 ms	1,095
gzip -6 (referencia)	88040 B	307200	3,489	81 ms	1,000
FGK (propio)	176599 B	307200	1,740	1314 ms	0,305

W > 1: mejor compromiso que gzip. W < 1: peor. Brotli supera a gzip; FGK queda por debajo.

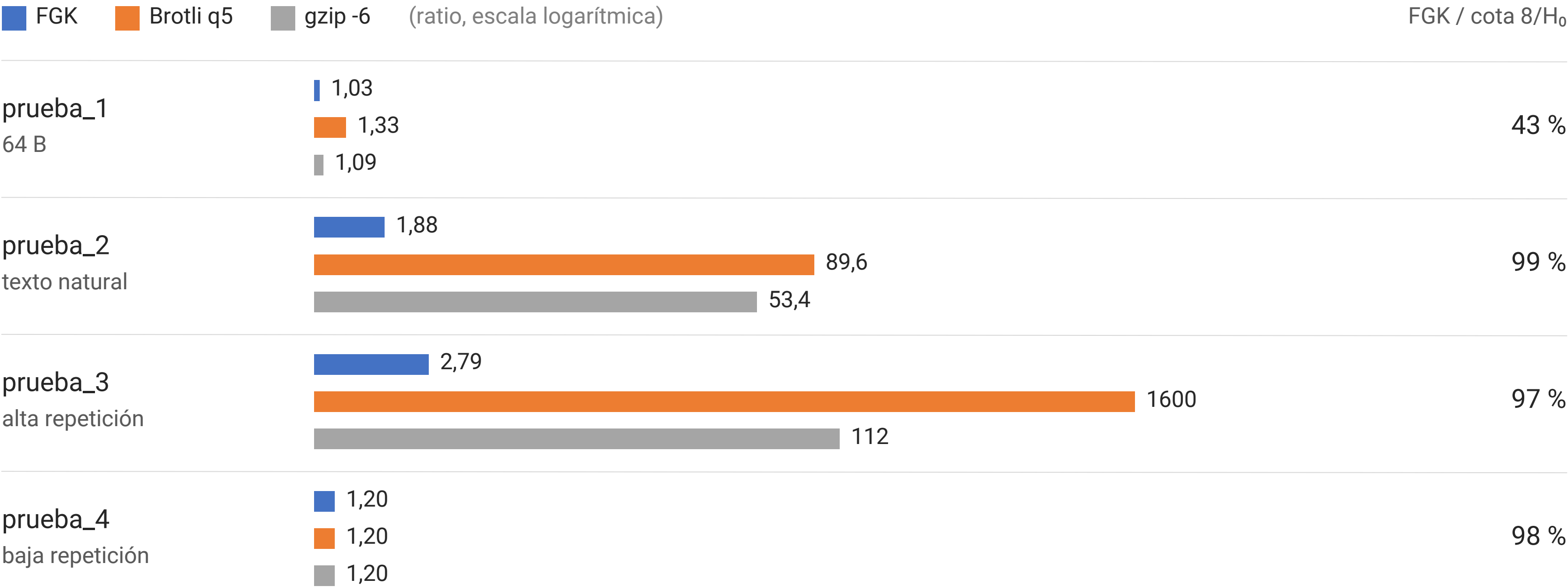
¿Por qué FGK obtiene $W = 0,305$?

El Weissman es un producto de dos factores, así que se puede separar:

- **Factor ratio:** $1,740 / 3,489 = 0,50$. Huffman dinámico es un modelo de orden 0: solo aprovecha la frecuencia de cada byte, no las repeticiones de cadenas.
- **Factor tiempo:** $\ln 81 / \ln 1314 = 0,61$. FGK es unas 16 veces más lento (Python puro contra C, más ~95 ms de arranque del intérprete), pero el logaritmo atenúa la diferencia.
- **Resultado:** $0,50 \times 0,61 \approx 0,305$.

Aunque FGK fuera tan rápido como gzip, W quedaría cerca de 0,5: el límite está en el modelo, no en la implementación.

Resultados según el tipo de archivo



Conclusiones

Recomendación

Para comprimir texto en general recomendamos Brotli q5: tuvo el mayor ratio en los 4 archivos y un Weissman global de 1,095.

Cuándo conviene FGK

Cuando se necesita una sola pasada (streaming) o los datos no tienen repeticiones largas. Solo necesita 20 bytes de cabecera.

Nuestro FGK

Funciona correctamente y llega al 97–99 % de su límite teórico, pero al ser de orden 0 no aprovecha las repeticiones.

Posibles mejoras

Modelo de contexto (Markov de orden 1 o PPM), una etapa LZ antes de Huffman, una implementación en C o la variante de Vitter.